The Safety Net With a Hole: Coverage Drift Between Failure States and Recovery States in a Self-Healing Workflow Runtime

The Safety Net With a Hole: Coverage Drift Between Failure States and Recovery States in a Self-Healing Workflow Runtime

Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: the July 2026 continuation-liveness investigation — a disproven fix, a read-only forensic redo, and a verified low-risk fix — and its August 2026 sequel (§8), a second stall in the same workload whose mechanism is the dual of the first; all commits, probes, and workflow-state records are in the Facetwork repository and its fleet's execution history.

Abstract

Leaderless workflow runtimes lean on *safety-net* recovery loops — periodic sweeps, reapers, watchdogs — that scan persisted state for work that has stalled and nudge it forward. Such a loop encodes, explicitly or not, a set of **states that need rescue**; correctness requires that set to equal the set of **states at which work can actually strand**. Nothing enforces the equality, and it drifts as the state machine grows. We report a production liveness bug that was exactly this drift: a workflow step that entered error-recovery (`CATCH_BEGIN`) could strand indefinitely because the catch states had never been added to the sweep's rescue set, even as they became genuinely-blocking states. The bug had defied diagnosis for months, manifesting as deep fan-out workflows that froze at their fan-in tail and required manual "re-seeding" to finish. We describe two things of independent interest: (1) a *disproven* first fix that reasoned from code to a plausible cause (optimistic-concurrency sequence regression) and, by changing write semantics on that hypothesis, converted a deadlock into a **livelock** (13,870 self-triggering recovery tasks in one run) before being reverted; and (2) the disciplined redo — a read-only forensic probe that froze a live stall, localized the frontier to a single catch-stranded step, and *confirmed* the mechanism by seeding one recovery task and watching the step advance in six seconds. The real fix is a read-query change with no write-semantics risk: a single canonical set of stall-states referenced by every sweep site. Verification on the workload that had never once completed without babysitting: it completed hands-off, zero re-seeds, identical output. A second stall in the same workload, root-caused three weeks later (§8), turns out to be the **dual** of this mode and sharpens the lesson: the stranding state was correctly covered, the sweep did select and process those steps, and the rescue still accomplished nothing, because the parked-step update was discarded before commit and the flag its re-queue path reads was never persisted at all. Coverage of the *selection* layer is necessary and not sufficient; a safety net must also be observable in its *effect*, since a loop that reports "resumed 68 steps" while committing nothing looks healthy in every log it writes. The transferable lessons are a named failure mode (*coverage drift*) with a structural remedy (one authoritative stall-state set), its dual (a rescue that runs and is a no-op), and a methodological rule the false start earned the hard way and the second stall confirmed twice more: **for liveness bugs, observe the frozen system before you mutate a write path.**

1. Introduction

A distributed workflow step in Facetwork advances through a persisted state machine, and *any* runner may perform the next transition by claiming a lightweight *continuation* task. Normally, completing one step generates continuations for the parents it unblocks, and the cascade walks itself to completion. When that cascade drops a link — a continuation that should have been created but wasn't, or a transition that never got triggered — a step can sit non-terminal with nothing pointing at it. The engine has a safety net for exactly this: a periodic **stuck-step sweep** that scans for steps in known-blocking states and resumes them.

For months, deep fan-out workflows (a continental "emergency-access atlas", tens of thousands of steps) stalled at their fan-in tail: a plateau of non-terminal steps, zero in-flight tasks, zero pending continuations, the runner reporting "running," no progress. The operational workaround was a *reseed driver* — a babysitting loop that manually re-enqueued continuations for every non-terminal step, advancing the workflow one generation per push. It worked, but it meant no deep run ever finished unattended, and the sweep — the very mechanism designed to prevent this — visibly failed to help.

This paper is the account of finding out why. It has an unusual shape because the investigation did: the first fix was *wrong in an instructive way*, and the second succeeded only by abandoning reasoning-from-code for reading-from-the-running-system. We contribute:

1. **A named failure mode** (§6): *coverage drift* between the set of states at which work strands and the set a safety-net loop rescues — a silent, growing liveness hole in any self-healing recovery system.
2. **A cautionary case** (§4): a plausible, code-reasoned fix that changed write semantics and made the failure strictly worse (deadlock → livelock), with the mechanism of the amplification.
3. **A read-only forensic method** (§5) that localized and *confirmed* the real cause without perturbing the system, and the six-second experiment that settled it.
4. **A low-risk structural fix and its measured effect** (§7): one authoritative stall-state set; before/after on the same workload.
5. **The dual failure mode** (§8): a later stall in the same workload where coverage was correct and the rescue ran anyway to no effect — with the obligation it implies, that a self-healing loop be instrumented by the transitions it *commits* rather than the rescues it *attempts*.

2. Background

Continuations and the cascade. Each step lives in the database. When a step reaches a terminal state, the iteration that completed it generates continuation tasks (`_fw_continue`) for the parent blocks it unblocks; a runner claims each and performs the next transition. This is the normal engine of progress — no sweep involved.

The stuck-step sweep. Because the cascade can drop links (a commit race, a crash between generating a transition and enqueuing its continuation, a runner dying mid-iteration), each runner periodically runs a sweep:

1. *Select* workflows that have any step in a blocking state (`get_pending_resume_workflow_ids`).
2. For each, *fetch* the stuck steps (`get_stuck_steps_for_workflow`) and resume them (an idempotent, deterministic re-derivation).

Both queries filter on a hardcoded set of states. The sweep runs on a jittered ~5-minute cadence and is deliberately rate-limited.

The catch clause. Error recovery (`catch`) routes a failed step into a recovery sub-machine: the step transitions to `CATCH_BEGIN`, a recovery sub-block is created (`CATCH_CONTINUE` observes it to completion), and its yield supplies the step's results. Catch was a comparatively recent feature; it introduced two new genuinely-blocking states, `CATCH_BEGIN` and `CATCH_CONTINUE`, at which a step waits for the recovery sub-machine to be created and to finish. *(That these states themselves took a multi-round campaign to make work distributed is a separate paper; here they are simply new blocking states in the machine.)*

3. The symptom, precisely

The stall signature: a workflow with N non-terminal steps, **zero** pending or running tasks (including zero `_fw_continue`), runner state "running," no step advancing for many minutes. It was scale-sensitive — small runs eventually crawled to completion, large ones did not within any patient timeout — which is the fingerprint of a recovery mechanism that is *present but too slow or too partial* rather than absent.

Two facts framed the search. First, the sweep exists and covers block-level blocking states, so a wholesale "the sweep doesn't run" theory was wrong — it ran, and workflows *were* being selected by it. Second, re-seeding continuations by hand always worked, which proved the stuck steps were *processable* — they were not deadlocked on missing data or a genuine dependency, merely un-triggered. The question was why the cascade dropped the trigger and why the sweep did not supply it.

4. The false start: a write-path fix that made it worse

Reasoning from the code, a plausible culprit presented itself. Steps carry an optimistic-concurrency *sequence* for their persisted version; the batch commit advances a step only if the database copy is strictly behind (compare-and-set). Fleet logs showed recurring CAS conflicts on the *same* step IDs at sweep cadence — "our sequence=1, DB already at >= it" — dropping writes. And a direct-save code path wrote the caller's in-memory sequence verbatim, which could *regress* the counter. The hypothesis wrote itself: a stale save resets the CAS baseline, racing writers fight to re-advance from zero, and legitimate transitions get dropped. The observed CAS-conflict warnings were real, so the hypothesis felt confirmed.

The fix followed the hypothesis: make the sequence server-authoritative (an atomic `$inc` on save, never trusting the caller), and — since a dropped transition otherwise strands its parents — enqueue a *self-wake continuation* when a batch write lost the CAS, so "someone re-derives from the winning state."

It made the bug dramatically worse. On the verification run the workflow was *more* stuck, not less, and the database held **13,870** self-wake continuations for one run. Two amplifying mechanisms, both consequences of touching write semantics:

- **Dual sequence authority.** The batch-commit path still managed the sequence in its CAS; the new `$inc` managed it independently on every direct save. Two writers advancing the same counter by different rules produced *more* CAS conflicts, not fewer — the exact metric the fix was meant to reduce.
- **A self-triggering recovery loop.** Each CAS conflict now enqueued a continuation; claiming it re-derived and re-attempted the write; which conflicted again against the still-racing counter; which enqueued another. A deadlock (nothing progressing) had become a **livelock** (a great deal progressing, none of it forward). The de-duplication guard (one pending wake per step) capped the *instantaneous* count but not the *cumulative* churn, because each wake completed before the next conflict created its successor.

We reverted the entire change. The lesson, which §5 then acted on: **the CAS conflicts were a real signal, but the hypothesis mistook a symptom for the mechanism, and validating it required changing write semantics — the one category of change that can convert a stall into a storm.** A liveness bug in a concurrent system is precisely where reasoning-from-code is least trustworthy, because the failure lives in interleavings the code does not locally reveal.

5. Read-only forensics

The redo obeyed a single rule: *change nothing until the frozen system has been read.* We built a read-only probe that, given a stalled workflow, reports its structure without mutation:

- Every non-terminal step by state.
- The **frontier**: non-terminal steps all of whose children are terminal — the deepest stuck points, the ones that *should* be advancing.
- For each frontier step, whether any task or continuation targets it.
- A **dry run**: seed a throwaway in-memory store with the workflow's steps and ask the evaluator to process a frontier step there, observing whether it would advance — against a copy, so the real database is untouched.

To catch a stall before the slow sweep erased it, we triggered a modest run (six regions, one designed to fail so the catch path fired) and polled at twelve-second resolution, freezing on the plateau. The probe's output was unambiguous:

```
non-terminal: 6  (2 blocks.Continue, 3 execution.Continue, 1 catch.Begin)
in-flight tasks: 1 (a lingering root task); pending _fw_continue: 0
frontier: 1 step, state = catch.Begin, seq 0, age 154s,
          4 children (all terminal), live tasks targeting it: NONE
```

The frontier was a single step at `catch.Begin` — entered catch recovery, never processed, no continuation, no task, stuck two and a half minutes and counting. The five other non-terminal steps were its ancestors, at block-continue states, each correctly waiting on it.

Rather than trust the dry run alone (its AST-loading path hit an unrelated snag), we ran the definitive experiment, and it is the crux of the paper. We seeded **one** continuation task pointed at the frozen step — adding a task, not changing any write path — and watched:

```
BEFORE: catch.Begin, seq 0
+6s:    ADVANCED -> catch.Continue
```

Six seconds. The step was fully processable the entire time; it had simply never been triggered, and nothing in the safety net was going to trigger it. That single observation converted the investigation from hypothesis to fact: the stall is not a data problem, not a race that corrupts state, not the CAS issue the first fix chased — it is a step in a blocking state that *no mechanism is looking for*.

6. The root cause: coverage drift

With the frontier state known, the cause was a two-line read of the code. The sweep's queries filter on a hardcoded state set:

```
{ EVENT_TRANSMIT, STATEMENT_BLOCKS_BEGIN, STATEMENT_BLOCKS_CONTINUE,
  BLOCK_EXECUTION_BEGIN, BLOCK_EXECUTION_CONTINUE }
```

`CATCH_BEGIN` and `CATCH_CONTINUE` are absent. They are genuinely-blocking states — a step waits at them for the recovery sub-machine — but they were added to the state machine (with the catch feature) and *never added here*. The safety net had a hole precisely the shape of error recovery.

The scale-sensitivity and the "sweep runs but doesn't help" paradox both fall out of this. The stalled workflow *was* selected for sweeping — because its block-continue ancestors are covered states. But `get_stuck_steps_for_workflow` then returned only those ancestors, never the `catch.Begin` frontier. So every sweep faithfully resumed the ancestors, which re-checked their children, found the catch step still non-terminal, and stayed put — the sweep resumed the wrong steps forever while the real blocker sat untouched. The step recovered only if some *other* path (a parent re-evaluation happening to mark it dirty and generate a continuation) reached it — indirect, incidental, and paced far slower than the fan-in produced new catch frontiers. Small runs had few enough frontiers that the incidental path eventually cleared them; large runs produced them faster than they cleared.

We name this **coverage drift**: a self-healing recovery loop carries a set of states it rescues, the system carries a growing set of states at which work can strand, and nothing enforces that the first covers the second. Each new blocking state in the machine is a silent opportunity to open a liveness hole, and the hole is invisible to tests (the recovery loop is rarely unit-tested against every state) and to code review (the two sets live in different files from the state machine that should govern them).

7. The fix and its verification

The fix is deliberately the *opposite* of the reverted one in risk profile: no write semantics, no new task creation, no feedback path. We defined the stall-state set **once**, canonically, next to the state machine it derives from:

```
STUCK_STEP_STATES = ( EVENT_TRANSMIT, ...block states...,
                      CATCH_BEGIN, CATCH_CONTINUE )
```

and referenced it from every sweep site in both persistence backends. The sweep now fetches and resumes catch-stranded steps directly, via its existing, idempotent resume path. Defining the set in one place is the structural remedy for coverage drift: adding a blocking state to the machine now has one obvious place to register it, and the two persistence implementations can no longer disagree (a divergence that, in this codebase, has independently caused other coordination bugs). Parity tests assert both backends surface catch-stranded steps identically.

Verification was the workload that had never once completed without babysitting: the full 25-region atlas (catch exercised by a failed region), run with the reseed driver *removed*. It completed **hands-off in 18 minutes** — runner completed, zero reseed-driver tasks, zero dead-letters, rankings byte-identical to every correct prior run. The progress trace shows the mechanism working: the tail plateaued at three non-terminal steps for about five minutes, then one for about five more, then finished — each plateau a catch frontier waiting for the next jittered sweep cycle to resume it, `active=0` throughout confirming the recovery came from the *sweep*, not from re-seeding. Against the prior baseline (six to twenty manual reseed generations plus operator endgame nudges, every deep run), the difference is categorical: from never-unattended to unattended.

The residual is honest and small: a catch frontier waits up to one sweep cycle (~5 min) before recovery — latency, not a stall. Eliminating it turned out to be non-trivial, which is itself instructive. The obvious follow-up — seed a continuation for the catch step at catch-entry, so it never depends on the sweep — was attempted twice and reverted twice, each attempt landing on a different property of the same iteration model. Seeding it where updated steps are enumerated failed because the continuation generator, on the path that actually runs, receives a reset change set and derives targets only from a dirty-block set. Marking the catch step's own id into that dirty set failed because the iteration boundary subtracts already-processed steps from its target set, and the step was just processed *into* the catch state. The genuine fix must carry self-reprocess states across the processed-steps subtraction — a change to the core loop whose exact processed/dirty/push interplay produced the catch defects of the sibling parity-gap paper. We deferred it: the payoff is tail latency on an already-correct, hands-off system, and the blast radius is the most delicate loop in the engine. The episode reprises the paper's own lesson at smaller scale — the iteration model, like the coordination substrate, does not reveal its behavior to local reading; both times the fix "obviously" belonged somewhere it did nothing, and only running the system showed it.

8. The second variant: a rescue that ran and did nothing

§9 of the first version of this paper left a question open. Earlier stalls in this workload's history were described in terms of block-level `Begin` states rather than catch states, and we declined to claim they shared the catch-coverage mechanism. They did not. Three weeks later the block-level variant was reproduced, root-caused, and fixed, and its mechanism is the **dual** of coverage drift — instructive precisely because the remedy of §7 would never have caught it.

8.1 Symptom

The signature was familiar and the diagnosis was not. Leaf steps — the innermost work of a wide `foreach` — parked at `statement.blocks.Begin` *after their handler task had completed successfully*. The work was done; only the transition onward was missing. Unbounded, this was survivable: a 3,167-way county fan-out stranded roughly a tenth of its leaves and still finished, because the tail eventually cleared. Bounding the fan-out width made it fatal. Under `foreach … limit N`, stranded leaves hold window slots that are never released, so admission stops: runs froze at 628 of 3,167, and again at 1,368. The width cap did not cause the bug; it converted a slow tail into a hard stall, which is what a bulkhead does to any leak of concurrency permits.

8.2 Why §7's remedy did not apply

`statement.blocks.Begin` was **in** the canonical stall-state set. The sweep selected these workflows, `get_actionable_steps_by_workflow` returned the leaves themselves, and the sweep called its resume path on them. Coverage was correct. The rescue nevertheless accomplished nothing, and did so silently.

Two independent defects were both required:

1. **The deferral was discarded.** A handler state that cannot proceed parks its step by returning "stay, and push me for re-queue". The evaluator's branch for *changed but not advanced* steps was guarded on `not result.continue_processing` — which is exactly false for a deferral. The step update was dropped: the push flag never reached persistence, no parent was marked dirty, and the iteration reported no changes. A sweep over such a step therefore committed **nothing**, which is why the sweep appeared inert on precisely the steps it was correctly selecting.
2. **The flag was not persisted at all.** The re-queue path that *is* designed to rescue a parked step — a scan of a dirtied container's children for the push flag — reads that flag back from the store. It was absent from the persisted step document, so the scan read false for every step, forever. The designed rescue was dead code on the production backend independently of (1).

Either defect alone hides the other. Fixing the selection set — the remedy this paper proposes — would have changed nothing here, because selection was never the problem.

8.3 The dual failure mode

Coverage drift is a defect of the **selection** layer: the loop does not *look at* the states where work strands. This is a defect of the **effect** layer: the loop looks at the right state, runs, and has no effect. Both present identically to an operator — a safety net that runs on schedule while work sits still — and the second is the more insidious, because every observable signal says the net is working. The sweep logged the workflow, it logged the step, it reported success. Only the absence of a *write* betrayed it.

The generalisation for any self-healing loop is a second obligation beside coverage: **a rescue must be observable in its effect, not merely in its execution**. A loop that cannot distinguish "resumed 68 steps" from "resumed 68 steps and committed nothing" cannot be trusted by the operator reading its logs. Counting committed state transitions, not attempted rescues, would have surfaced this in an afternoon rather than across months and two failed fixes.

8.4 Method, again

Two fixes preceded the correct one, and both failed the way §4's fix failed: a plausible mechanism reasoned from reading code, shipped through a full image bake and fleet rollout, changing nothing. One targeted a window-refill nudge that sat behind the very resume that was dying before reaching it; its warning never once appeared in a log. The other restored an AST that was never missing.

The correct diagnosis took one run of a twenty-line probe: stub the state changer to return a deferral, call the evaluator's step-processing function directly, print what was committed. It reported the persisted flag false, the updated-steps list empty, and the dirty set empty — the entire mechanism, in a single output, after roughly an hour of code reading had produced only stories. This is §4's lesson recurring with a sharper edge: it is not merely that one must observe before mutating a write path, but that the observation is usually *cheap and available* at the moment one is tempted to theorise.

8.5 Verification

The fix persists the parked step on the false-to-true edge of the flag only — idempotent, so re-sweeping an already-parked step remains a no-op rather than a per-sweep write storm — and round-trips the flag through the store. Eight regression tests cover both persistence backends and were confirmed to **fail against the pre-fix code** before being accepted, including an end-to-end one: a sibling landing must re-queue a parked step, which pre-fix ends after a single iteration with the parked step untouched.

The behavioural criterion this workload had never met was that a *capped* fan-out survive a runner restart unattended. On the deployed fix: sixty iterations at width six, the runner killed with `SIGKILL` mid-flight with one task in flight and twenty-nine iterations not yet admitted, then restarted once and left alone. It completed in about 135 seconds — 61 of 61 tasks, none dead-lettered, exactly one task re-claimed by the orphan reaper, the one that had been killed. That is the shape that previously stalled.

We scope this deliberately. Sixty iterations on one runner does not exercise the contention, sweep cadence and multi-runner re-claim that made stranding common at 3,167 iterations across twenty-one runners, and the national atlas has not been re-run capped since the fix. The mechanism is established and the restart criterion is met; the scale claim is not yet earned.

9. Related work and discussion

Self-healing and safety nets. Reconciliation loops (Kubernetes controllers), reapers, and watchdogs all encode a target set of unhealthy states to act on. The coverage-drift failure mode is latent in all of them: the target set is authored once and the state space grows around it. Our contribution is to name the mode and prescribe the structural remedy — a single authoritative set co-located with the state machine — rather than the usual per-incident patch that re-opens on the next new state.

Liveness testing. Liveness (something good eventually happens) is notoriously harder to test than safety (nothing bad happens); a missing recovery transition produces no error, no crash, no failed assertion — only the absence of progress, which no unit test naturally waits for. Model checkers (TLA+) can express and check liveness under fairness, but a spec of this engine would model the sweep as "eventually resumes any stuck step" — abstracting away the very hardcoded state set whose incompleteness was the bug. As with the sibling parity-gap finding, the defect lived below the altitude at which a reasonable model operates.

Reasoning vs. observation. The disproven first fix is the paper's methodological core. It was not careless: it was reasoned from real evidence (the CAS conflicts) to a plausible mechanism, and its author (a human–AI pair, per the thesis's authorship addendum) was confident. It failed because concurrent-liveness mechanisms are not locally legible in code — the fault is in interleavings and in the *absence* of an action, neither visible at a call site. The read-only probe succeeded because it read the *effect* (a step frozen in a specific state with nothing pointing at it) rather than inferring the *cause*, and the single-continuation experiment confirmed the mechanism by perturbation-of-one. The rule we draw — observe the frozen system before mutating a write path — is not a counsel of caution but of *evidence order*: in a concurrent system the state is more truthful than the source.

10. Threats to validity and scope

The §7 fix is verified for the **catch-frontier** variant of the stall. The block-level `Begin` variant, whose mechanism this paper originally declined to claim, was established separately (§8) and proved to be *distinct*: coverage was correct there and the defect lay in the rescue's effect. That resolves the open question but also bounds the §7 remedy — a canonical stall-state set fixes selection, and nothing about selection would have caught §8. Readers should treat the two as complementary obligations rather than one fix.

The §8 verification carries its own limit, stated there and repeated here: a sixty-iteration capped fan-out on a single runner meets the restart criterion but does not reproduce the scale at which the stall was originally common (3,167 iterations, twenty-one runners); that run has not been repeated capped since the fix. We report one workload on one four-host fleet; the hands-off result is a single (repeatable) observation, not a distribution. The ~5-minute residual latency means the fix restores *correctness and unattendedness*, not *speed* — a system needing low fan-in-tail latency would want the catch-entry continuation follow-up. Finally, coverage drift is argued as a general mode from one instance; we claim the mechanism and remedy transfer, not a frequency across systems.

11. Conclusion

A workflow froze at its tail for months, and the safety net built to prevent exactly that watched it freeze — because the net's list of states-to-rescue had never grown to include the states error-recovery introduced. The first fix, reasoned confidently from a real symptom, changed write semantics and turned the freeze into a storm; the second, forbidden from touching write paths until it had read the frozen system, found a single stuck step, proved it processable in six seconds, and closed the hole with one canonical list. The workload that had never finished unattended now does.

A second stall in the same workload then made the same point from the other side. There the state *was* covered, the sweep *did* process it, and the rescue still did nothing — the parked-step write was discarded before commit and the flag the designed re-queue path reads had never been persisted at all. Two more code-reasoned fixes died against it before a twenty-line probe settled it in one run. A capped fan-out now survives a `SIGKILL` mid-flight and finishes unattended.

The bugs were small; the lessons are not. *Coverage drift* is a liveness hole latent in every self-healing loop, and it has a dual that is harder to see: a loop can select the right work, run on schedule, report success, and commit nothing — so a safety net must be instrumented by the state transitions it *commits*, not the rescues it attempts. And across all three failed fixes in this history, *observe before you mutate* was the evidence order that concurrent-liveness debugging demanded and that reading code, however carefully, never supplied.

Artifacts: disproven fix `a4c8ad4` (reverted `b13b927`); the read-only probe (`docs/thesis/artifacts/stall_probe.py`) and the single-continuation confirmation; the fix `077a264` (canonical `STUCK_STEP_STATES` in `states.py`, both persistence backends, parity tests `tests/runtime/test_liveness_sweep.py`); engineering doctrine `docs/architecture/lessons-learned.md` §25; the hands-off verification run of `osm.emergency.flows.ContinentalEmergencyAtlas` on fleet v96 and its persisted records.